

## Informe Preliminar : Desafío 1

**Materia : Informática II**

**Semestre 2026-2**

**Integrantes :**

**Santiago Osorio Fernandez  
Juan José Guarín Atehortua**

**Universidad de Antioquia**

### Contextualización

El presente desafío aborda el análisis y diseño de un juego de alineación de elementos tipo Match-3. El desafío consta de las siguientes restricciones: No se deben utilizar clases ni otras librerías que simplifiquen la lógica y análisis de la manipulación de datos en memoria; se deben manipular los datos a nivel de bits con arreglos dinámicos, se deben evaluar reglas de adyacencia de forma recurrente (cascadas), reestructurar las dimensiones del arreglo en tiempo real, además se debe programar pensando en el desarrollo de una micro arquitectura para sistemas con poca capacidad de memoria, teniendo siempre en cuenta el uso eficiente de la misma : por cada ficha se utilizarán 3 bits de almacenamiento en lugar de un byte.

La solución a este desafío se programará en C + + bajo el paradigma de la programación estructurada modular, utilizando submódulos de bajo nivel interconectados mediante el paso de referencias y punteros directos a memoria. Esto con el fin de mantener una arquitectura altamente eficiente y con el uso mínimo de recursos. El juego final se ejecutará con Qt creator y la salida de datos se hará en la consola del mismo programa.

### Análisis y diseño :

#### 1. Definición del Espacio de Datos (Modelamiento de Fichas)

El núcleo de la solución empieza por decidir cómo vamos a representar las fichas del juego dentro de la memoria del sistema. con 3 bits podemos representar hasta 8 estados diferentes (desde 000 hasta 111). En este caso utilizaremos **unsigned char** como tipo de dato ya que es la unidad mínima que nos permite C++ (1 byte) y es la que mejor se adapta al trabajo con bits, le quitaremos el signo para evitar que C++ agregue signos a los valores numéricos asignados a las fichas, se deberá crear tres funciones una para crear las fichas con valores aleatorios, una para ubicar la posición de una ficha dentro de la memoria y otra para modificar el valor de la ficha.

#### 2. Análisis del Direccionamiento Físico (Mapeo Bidimensional a Unidimensional)

Esta es la parte más importante de la matemática del juego. El jugador ve un tablero de filas (F) y Columnas (C) (una matriz de dos dimensiones). Sin embargo, la memoria de la computadora es una línea continua de bytes. Nuestro trabajo es crear un puente matemático para convertir una coordenada (fila, columna) del jugador en una ubicación exacta dentro de nuestro arreglo de bytes.

Problema: El tablero (FxC fichas) se guarda como una secuencia de bits. Para encontrar una ficha, debemos calcular dónde empiezan sus 3 bits en esa secuencia.

La solución: Usamos las fórmulas dadas en el enunciado para navegar del tablero del jugador al espacio físico de almacenamiento, que es la memoria del computador.

Además agregaremos una fórmula más para identificar en qué posición se encuentra el primer bit de una ficha dentro del Byte cuyo valor estará entre 0 y 7:

**desplazamiento** = bit\_inicio % 8

#### **Cálculo de la Memoria Necesaria:**

El tablero tiene  $F * C$  posiciones, y cada una usa 3 bits. Por lo tanto, el total de bits es  $3 * F * C$ . Como la memoria se reserva en bytes, debemos calcular cuántos bytes se necesitan para guardar todos esos bits. Si el total de bits no es múltiplo de 8, debemos redondear hacia arriba para tener espacio para todos.

$$\text{bytes\_necesarios} = (3 * F * C + 7) / 8$$

Es la cantidad mínima exacta necesaria para almacenar todos los bits de las fichas en el tablero. En estricto cumplimiento con los requerimientos del desafío, la arquitectura de software alineará la secuencia de datos válidos comenzando desde el bit menos significativo (bit 0) del primer byte, propagándose de forma continua.

En consecuencia, los bits remanentes o "inválidos" se concentrarán de forma natural en las posiciones de mayor peso (bits más significativos) del último byte reservado.

### **3. Fronteras de Memoria**

Teniendo en cuenta las restricciones del enunciado : La repartición de los bits en la memoria física no coincide con los espacios de memoria de los bytes, esto genera tres casos que debemos resolver para poder identificar en que byte se encuentra una ficha dentro de la memoria, anteriormente se definieron las fórmulas para identificar el bit inicial de una ficha y el desplazamiento que nos indicará si una ficha está enteramente contenida en un byte o fraccionada en dos bytes.

#### **Caso 1 : Los tres bits se encuentran dentro de un mismo byte :**

En este caso la posición inicial del primer bit de la ficha se encuentra entre los valores 0 a 5 querra decir que la ficha se encuentra enteramente en el byte N. así que para encontrar los tres bits de la ficha bastará con desplazarnos el número de veces indicado por el "desplazamiento", tomar el bit actual más los dos siguientes y luego aislar los bits de la ficha con una máscara.

#### **Caso 2. Ficha repartida entre dos bytes (2+1) :**

Este caso ocurrirá cuando el desplazamiento adquiera el valor de 6. Esto nos indica que los dos primeros bits se encontraran en el byte N y el último bit se encontrará en el MSB del byte N+1. En este caso debemos recuperar ambos fragmentos de la ficha con manipulación de máscaras en ambos bytes y luego unir los bits en un solo dato. El reto está en hacer que los bits conserven su posición original para no alterar el valor de la ficha.

#### **Caso 3. Ficha repartida entre dos bytes (1+2) :**

Este caso ocurrirá cuando el desplazamiento adquiera el valor de 7. Esto nos indica que el primer bit se encontrará en el byte N y los dos últimos bits de la ficha se encontraran en el byte N+1. El proceso de recuperación de los fragmentos es similar al descrito en el caso anterior.

Para las secciones **1, 2 y 3** se emplearán las siguientes **funciones** :

**Creación del tablero, destrucción del tablero, obtención ficha y modificación ficha.**

### **4. Algoritmo de gravedad y Lógica de combinaciones válidas,**

#### **Algoritmo de gravedad:**

Después de que el jugador elimina una ficha, el tablero entra en un estado de "inestabilidad". El espacio de la ficha eliminada se marcará en estado vacío (110) y en

seguida se debe llenar por “gravedad” es decir la ficha en la posición inmediatamente superior (índice - columnas) debe ocupar el lugar de la ficha que el usuario eliminó y así sucesivamente hasta llegar a la fila cero donde se deberá generar una nueva ficha siguiendo los parámetros de creación aleatoria uniforme de una ficha.

En el caso de fichas eliminadas por las combinaciones generadas, se hará un barrido de todas las columnas de abajo hacia arriba identificando las posiciones vacías (110) en este caso se buscará por medio de una iteración el primer elemento no vacío en la columna y se traerá esta ficha a la posición vacía inicial, el proceso se repetirá hasta que todas las fichas por encima están vacías, una vez este proceso realizado en todas las columnas, se procederá a generar nuevas fichas.

**Lógica de combinaciones válidas:** El tablero se escaneará de manera lineal para cada fila y cada columna, para la detección de combinaciones válidas debemos definir la adyacencia vertical y horizontal.

**La adyacencia vertical:** se buscará en cada columna si la ficha es igual a la que está inmediatamente debajo de ella, es decir, al índice de la ficha actual se le sumará sucesivamente el número de columnas activas del tablero ( $\text{índice} + n \times \text{Columnas}$ ). Se registrará el conteo de las fichas iguales, si después de una secuencia de coincidencias el contador es igual o mayor a tres, todas estas coordenadas se identificarán como activas en un arreglo dinámico temporal para eliminarlas en una etapa posterior.

**Adyacencia horizontal:** En este caso se recorren las filas mediante incrementos unitarios ( $\text{índice} + n$ ). Para evitar crear una falsa coincidencia entre la última ficha de la fila actual y la primera de la siguiente se verificará que el cociente de ( $\text{índice} / \text{Columnas}$ ) sea igual a lo largo de la evaluación de la fila, luego de este proceso, se verificará si las fichas son iguales, al igual que en el eje vertical, si el contador de coincidencias es mayor o igual a tres, se enviarán los índices al arreglo temporal para su posterior eliminación.

**Eliminación de fichas:** Una vez todas las fichas que se eliminarán sean identificadas en un arreglo externo se procede a eliminarlas físicamente de la memoria, el sistema debe consultar el arreglo temporal y modificar los bits de las posiciones marcadas agregándoles el valor del estado vacío (110) así nos aseguramos que las combinaciones en cruz o en L y sus variaciones se eliminen del tablero correctamente.

Se crearán las siguientes funciones: **generación de las fichas según una distribución uniforme, función de detección de combinaciones, función de gravedad y una función de eliminación de fichas**

## 5. Reacciones en Cadena y estabilización del tablero.

Luego de que se eliminen las fichas por combinaciones válidas y el algoritmo de gravedad se haya ejecutado, se pueden formar nuevas combinaciones por dicha reorganización.

### El Bucle de Equilibrio Dinámico:

El análisis que tomamos para solucionar esto es usar un proceso iterativo, un bucle que se repite hasta que el tablero se “estabilice”. El flujo sería el siguiente:

1. **Eliminación del jugador:** El usuario indica una ficha a eliminar del tablero.
2. **Bucle infinito (controlador):** Estamos en un ciclo while (true).
3. **Paso 1 - Gravedad:** Dentro del bucle, primero aplicamos el algoritmo de gravedad. Esto hace que todas las fichas caigan hacia abajo ocupando los huecos que hayan.
4. **Paso 2 - Relleno:** Una vez que las fichas han caído, los espacios que quedan vacíos en la parte superior se llenan de fichas nuevas.
5. **Paso 3 - Detección:** Escaneamos todo el tablero de nuevo para ver si se han formado nuevas combinaciones de 3 o más fichas iguales.
6. **Decisión:**
  - Si no se detectan nuevas combinaciones: El tablero ha llegado al equilibrio. El juego está estable. Salimos del bucle y le devolvemos el control al jugador.

[illegible]